CHAPTER

Introduction

C7729_ch01.indd 1 03/01/11 8:54 AM

CHAPTER 1

How we communicate influences how we think, and vice versa. Similarly, how we program computers influences how we think about computation, and vice versa. Over the last several decades, programmers have, collectively, accumulated a great deal of experience in the design and use of programming languages. Although we still don't completely understand all aspects of the design of programming languages, the basic principles and concepts now belong to the fundamental body of knowledge of computer science. A study of these principles is as essential to the programmer and computer scientist as the knowledge of a particular programming language such as C or Java. Without this knowledge it is impossible to gain the needed perspective and insight into the effect that programming languages and their design have on the way that we solve problems with computers and the ways that we think about computers and computation.

It is the goal of this text to introduce the major principles and concepts underlying programming languages. Although this book does not give a survey of programming languages, specific languages are used as examples and illustrations of major concepts. These languages include C, C++, Ada, Java, Python, Haskell, Scheme, and Prolog, among others. You do not need to be familiar with all of these languages in order to understand the language concepts being illustrated. However, you should be experienced with at least one programming language and have some general knowledge of data structures, algorithms, and computational processes.

In this chapter, we will introduce the basic notions of programming languages and outline some of the basic concepts. Figure 1.1 shows a rough timeline for the creation of several of the major programming languages that appear in our discussion. Note that some of the languages are embedded in a family tree, indicating their evolutionary relationships.

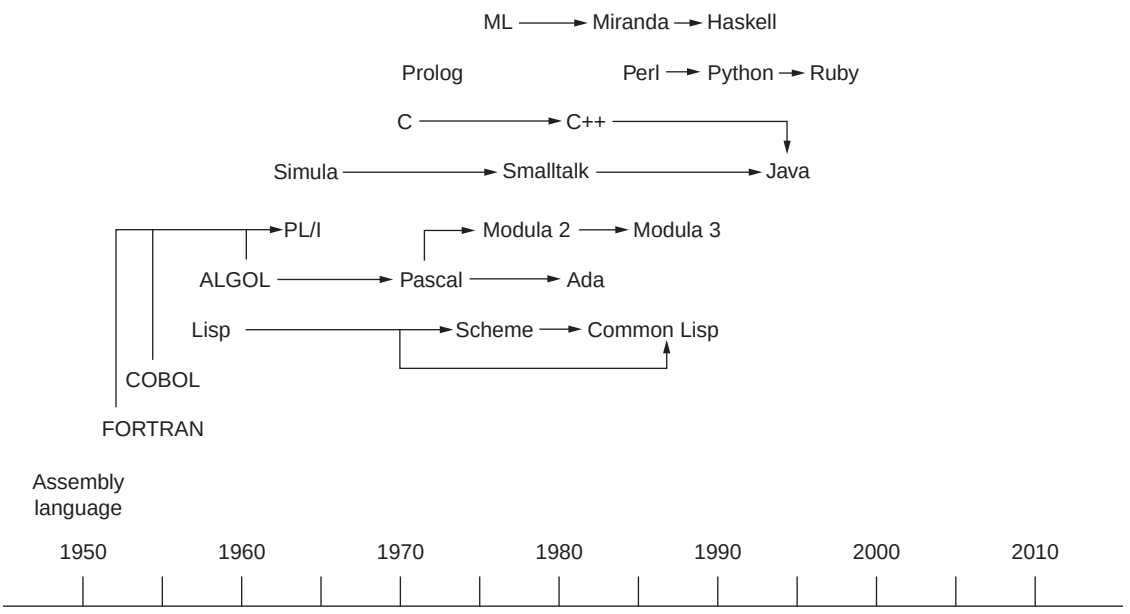

Figure 1.1 A programming language timeline

1.1 The Origins of Programming Languages

A definition often advanced for a programming language is "a notation for communicating to a computer what we want it to do," but this definition is inadequate. Before the middle of the 1940s, computer operators "hardwired" their programs. That is, they set switches to adjust the internal wiring of a computer to perform the requested tasks. This effectively communicated to the computer what computations were desired, but programming, if it could be called that, consisted of the expensive and error-prone activity of taking down the hardware to restructure it. This section examines the origins and emergence of programming languages, which allowed computer users to solve problems without having to become hardware engineers.

1.1.1 Machine Language and the First Stored Programs

A major advance in computer design occurred in the late 1940s, when John von Neumann had the idea that a computer should be permanently hardwired with a small set of general-purpose operations [Schneider and Gersting, 2010]. The operator could then input into the computer a series of binary codes that would organize the basic hardware operations to solve more-specific problems. Instead of turning off the computer to reconfigure its circuits, the operator could flip switches to enter these codes, expressed in **machine language**, into computer memory. At this point, computer operators became the first true programmers, who developed software—the machine code—to solve problems with computers.

Figure 1.2 shows the code for a short machine language program for the LC-3 machine architecture [Patt and Patel, 2003].

```
0010001000000100
0010010000000100
0001011001000010
0011011000000011
1111000000100101
0000000000000101
0000000000000110
0000000000000000
```

Figure 1.2 A machine language program

In this program, each line of code contains 16 bits or binary digits. A line of 16 bits represents either a single machine language instruction or a single data value. The last three lines of code happen to represent data values—the integers 5, 6, and 0—using 16-bit twos complement notation. The first five lines of code represent program instructions. Program execution begins with the first line of code, which is fetched from memory, decoded (interpreted), and executed. Control then moves to the next line of code, and the process is repeated, until a special halt instruction is reached.

To decode or interpret an instruction, the programmer (and the hardware) must recognize the first 4 bits of the line of code as an **opcode**, which indicates the type of operation to be performed. The remaining 12 bits contain codes for the instruction's operands. The operand codes are either the numbers of machine registers or relate to the addresses of other data or instructions stored in memory. For example, the first instruction, `0010001000000100`, contains an opcode and codes for two operands. The opcode `0010` says, "copy a number from a memory location to a machine register" (machine registers are high-speed memory cells that hold data for arithmetic and logic computations). The number of the register, `001`, is found in the next 3 bits. The remaining 9 bits represent an integer offset from the address of the next instruction. During instruction execution, the machine adds this integer offset to the next instruction's address to obtain the address of the current instruction's second operand (remember that both instructions and data are stored in memory). In this case, the machine adds the binary number 100 (4 in binary) to the number 1 (the address of the next instruction) to obtain the binary number 101 (5 in binary), which is the address of the sixth line of code. The bits in this line of code, in turn, represent the number to be copied into the register.

We said earlier that execution stops when a halt instruction is reached. In our program example, that instruction is the fifth line of code, `1111000000100101`. The halt instruction prevents the machine from continuing to execute the lines of code below it, which represent data values rather than instructions for the program.

As you might expect, machine language programming is not for the meek. Despite the improvement on the earlier method of reconfiguring the hardware, programmers were still faced with the tedious and error-prone tasks of manually translating their designs for solutions to binary machine code and loading this code into computer memory.

1.1.2 Assembly Language, Symbolic Codes, and Software Tools

The early programmers realized that it would be a tremendous help to use mnemonic symbols for the instruction codes and memory locations, so they developed **assembly language** for this purpose.

This type of language relies on software tools to automate some of the tasks of the programmer. A program called an **assembler** translates the symbolic assembly language code to binary machine code. For example, let's say that the first instruction in the program of Figure 1.2 reads:

```
LD R1, FIRST
```

in assembly language. The mnemonic symbol `LD` (short for "load") translates to the binary opcode `0010` seen in line 1 of Figure 1.2. The symbols `R1` and `FIRST` translate to the register number `001` and the data address offset `000000100`, respectively. After translation, another program, called a **loader**, automatically loads the machine code for this instruction into computer memory.

Programmers also used a pair of new input devices—a keypunch machine to type their assembly language codes and a card reader to read the resulting punched cards into memory for the assembler. These two devices were the forerunners of today's software text editors. These new hardware and software tools made it much easier for programmers to develop and modify their programs. For example, to insert a new line of code between two existing lines of code, the programmer now could put a new card into the keypunch, enter the code, and insert the card into the stack of cards at the appropriate position. The assembler and loader would then update all of the address references in the program, a task that machine language programmers once had to perform manually. Moreover, the assembler was able to catch some errors, such as incorrect instruction formats and incorrect address calculations, which could not be discovered until run time in the pre-assembler era.

Figure 1.3 shows the machine language program of Figure 1.2 after it has been "disassembled" into the LC-3 assembly language. It is now possible for a human being to read what the program does. The program adds the numbers in the variables `FIRST` and `SECOND` and stores the result in the variable `SUM`. In this code, the symbolic labels `FIRST`, `SECOND`, and `SUM` name memory locations containing data, the labels `R1`, `R2`, and `R3` name machine registers, and the labels `LD`, `ADD`, `ST`, and `HALT` name opcodes. The program is also commented (the text following each semicolon) to clarify what it does for the human reader.

```
.ORIG x3000      ; Address (in hexadecimal) of the first instruction
LD  R1, FIRST    ; Copy the number in memory location FIRST to register R1
LD  R2, SECOND   ; Copy the number in memory location SECOND to register R2
ADD R3, R2, R1   ; Add the numbers in R1 and R2 and place the sum in
                 ; register R3
ST  R3, SUM      ; Copy the number in R3 to memory location SUM
HALT             ; Halt the program
FIRST  .FILL #5  ; Location FIRST contains decimal 5
SECOND .FILL #6  ; Location SECOND contains decimal 6
SUM    .BLKW #1  ; Location SUM (contains 0 by default)
.END             ; End of program
```

Figure 1.3 An assembly language program that adds two numbers

Although the use of mnemonic symbols represents an advance on binary machine codes, assembly language still has some shortcomings. The assembly language code in Figure 1.3 allows the programmer to represent the abstract mathematical idea, "Let `FIRST` be 5, `SECOND` be 6, and `SUM` be `FIRST + SECOND`" as a sequence of human-readable machine instructions. Many of these instructions must move

data from variables/memory locations to machine registers and back again, however; assembly language lacks the more powerful abstraction capability of conventional mathematical notation. An **abstraction** is a notation or way of expressing ideas that makes them concise, simple, and easy for the human mind to grasp. The philosopher/mathematician A. N. Whitehead emphasized the power of abstract notation in 1911: "By relieving the brain of all unnecessary work, a good notation sets it free to concentrate on more advanced problems. . . . Civilization advances by extending the number of important operations which we can perform without thinking about them." In the case of assembly language, the programmer must still do the hard work of translating the abstract ideas of a problem domain to the concrete and machine-dependent notation of a program.

A second major shortcoming of assembly language is due to the fact that each particular type of computer hardware architecture has its own machine language instruction set, and thus requires its own dialect of assembly language. Therefore, any assembly language program has to be rewritten to port it to different types of machines.

The first assembly languages appeared in the 1950s. They are still used today, whenever very low-level system tools must be written, or whenever code segments must be optimized by hand for efficiency. You will likely have exposure to assembly language programming if you take a course in computer organization, where the concepts and principles of machine architecture are explored.

1.1.3 FORTRAN and Algebraic Notation

Unlike assembly language, high-level languages, such as C, Java, and Python, support notations closer to the abstractions, such as algebraic expressions, used in mathematics and science. For example, the following code segment in C or Java is equivalent to the assembly language program for adding two numbers shown earlier:

```
int first = 5;
int second = 6;
int sum = first + second;
```

One of the precursors of these high-level languages was FORTRAN, an acronym for FORmula TRANslation language. John Backus developed FORTRAN in the early 1950s for a particular type of IBM computer. In some respects, early FORTRAN code was similar to assembly language. It reflected the architecture of a particular type of machine and lacked the structured control statements and data structures of later high-level languages. However, FORTRAN did appeal to scientists and engineers, who enjoyed its support for algebraic notation and floating-point numbers. The language has undergone numerous revisions in the last few decades, and now supports many of the features that are associated with other languages descending from its original version.

1.1.4 The ALGOL Family: Structured Abstractions and Machine Independence

Soon after FORTRAN was introduced, programmers realized that languages with still higher levels of abstraction would improve their ability to write concise, understandable instructions. Moreover, they wished to write these high-level instructions for different machine architectures with no changes. In the late 1950s, an international committee of computer scientists (which included John Backus) agreed on

a definition of a new language whose purpose was to satisfy both of these requirements. This language became ALGOL (an acronym for ALGOrithmic Language). Its first incarnation, ALGOL-60, was released in 1960.

ALGOL provided first of all a standard notation for computer scientists to *publish algorithms in journals*. As such, the language included notations for structured control statements for sequencing (`begin-end` blocks), loops (the `for` loop), and selection (the `if` and `if-else` statements). These types of statements have appeared in more or less the same form in every high-level language since. Likewise, elegant notations for expressing data of different numeric types (integer and float) as well as the array data structure were available. Finally, support for procedures, including recursive procedures, was provided. These structured abstractions, and more, are explored in detail later in this chapter and in later chapters of this book.

The ALGOL committee also achieved machine independence for program execution on computers by requiring that each type of hardware provide an ALGOL **compiler**. This program translated standard ALGOL programs to the machine code of a particular machine.

ALGOL was one of the first programming languages to receive a formal specification or definition. Its published report included a grammar that defined its features, both for the programmer who used it and for the compiler writer who translated it.

A very large number of high-level languages are descended from ALGOL. Niklaus Wirth created one of the early ones, Pascal, as a language for teaching programming in the 1970s. Another, Ada, was developed in the 1980s as a language for embedded applications for the U.S. Department of Defense. The designers of ALGOL's descendants typically added features for further structuring data and large units of code, as well as support for controlling access to these resources within a large program.

1.1.5 Computation Without the von Neumann Architecture

Although programs written in high-level languages became independent of particular makes and models of computers, these languages still echoed, at a higher level of abstraction, the underlying architecture of the von Neumann model of a machine. This model consists of an area of memory where both programs and data are stored and a separate central processing unit that sequentially executes instructions fetched from memory. Most modern programming languages still retain the flavor of this single processor model of computation. For the first five decades of computing (from 1950 to 2000), the improvements in processor speed (as expressed in Moore's Law, which states that hardware speeds increase by a factor of 2 every 18 months) and the increasing abstraction in programming languages supported the conversion of the industrial age into the information age. However, this steady progress in language abstraction and hardware performance eventually ran into separate roadblocks.

On the hardware side, engineers began, around the year 2005, to reach the limits of the improvements predicted by Moore's Law. Over the years, they had increased processor performance by shortening the distance between processor components, but as components were packed more tightly onto a processor chip, the amount of heat generated during execution increased. Engineers mitigated this problem by factoring some computations, such as floating-point arithmetic and graphics/image processing, out to dedicated processors, such as the math coprocessor introduced in the 1980s and the graphics processor first released in the 1990s. Within the last few years, most desktop and laptop computers have been built with multicore architectures. A multicore architecture divides the central processing unit (CPU) into two or more general-purpose processors, each with its own specialized memory, as well as memory that is shared among them. Although each "core" in a multicore processor is slower than the

CPU of a traditional single-processor machine, their collaboration to carry out computations in parallel can potentially break the roadblock presented by the limits of Moore's Law.

On the language side, despite the efforts of designers to provide higher levels of abstraction for von Neumann computing, two problems remained. First, the model of computation, which relied upon changes to the values of variables, continued to make very large programs difficult to debug and correct. Second, the single-processor model of computation, which assumes a sequence of instructions that share a single processor and memory space, cannot be easily mapped to the new hardware architectures, whose multiple CPUs execute in parallel. The solution to these problems is the insight that programming languages need not be based on any particular model of hardware, but need only support models of computation suitable for various styles of problem solving.

The mathematician Alonzo Church developed one such model of computation in the late 1930s. This model, called the **lambda calculus**, was based on the theory of recursive functions. In the late 1950s, John McCarthy, a computer scientist at M.I.T. and later at Stanford, created the programming language Lisp to construct programs using the functional model of computation. Although a Lisp interpreter translated Lisp code to machine code that actually ran on a von Neumann machine (the only kind of machine available at that time), there was nothing about the Lisp notation that entailed a von Neumann model of computation. We shall explore how this is the case in detail in later chapters. Meanwhile, researchers have developed languages modeled on other non–von Neumann models of computing. One such model is formal logic with automatic theorem proving. Another involves the interaction of objects via message passing. We examine these models of computing, which lend themselves to parallel processing, and the languages that implement them in later chapters.

1.2 Abstractions in Programming Languages

We have noted the essential role that abstraction plays in making programs easier for people to read. In this section, we briefly describe common abstractions that programming languages provide to express computation and give an indication of where they are studied in more detail in subsequent chapters. Programming language abstractions fall into two general categories: **data abstraction** and **control abstraction**. Data abstractions simplify for human users the behavior and attributes of data, such as numbers, character strings, and search trees. Control abstractions simplify properties of the transfer of control, that is, the modification of the execution path of a program based on the situation at hand. Examples of control abstractions are loops, conditional statements, and procedure calls.

Abstractions also are categorized in terms of **levels**, which can be viewed as measures of the amount of information contained (and hidden) in the abstraction. **Basic abstractions** collect the most localized machine information. **Structured abstractions** collect intermediate information about the structure of a program. **Unit abstractions** collect large-scale information in a program.

In the following sections, we classify common abstractions according to these levels of abstraction, for both data abstraction and control abstraction.

1.2.1 Data: Basic Abstractions

Basic data abstractions in programming languages hide the internal representation of common data values in a computer. For example, integer data values are often stored in a computer using a two's complement representation. On some machines, the integer value -64 is an abstraction of the 16-bit twos complement

value 1111111111000000. Similarly, a real or floating-point data value is usually provided, which hides the IEEE single- or double-precision machine representation of such numbers. These values are also called "primitive" or "atomic," because the programmer cannot normally access the component parts or bits of their internal representation [Patt and Patel, 2003].

Another basic data abstraction is the use of symbolic names to hide locations in computer memory that contain data values. Such named locations are called **variables**. The kind of data value is also given a name and is called a **data type**. Data types of basic data values are usually given names that are variations of their corresponding mathematical values, such as `int`, `double`, and `float`. Variables are given names and data types using a **declaration**, such as the Pascal:

```
var x : integer;
```

or the equivalent C declaration:

```
int x;
```

In this example, `x` is established as the name of a variable and is given the data type integer.

Finally, standard operations, such as addition and multiplication, on basic data types are also provided. Data types are studied in Chapter 8 and declarations in Chapter 7.

1.2.2 Data: Structured Abstractions

The **data structure** is the principal method for collecting related data values into a single unit. For example, an employee record may consist of a name, address, phone number, and salary, each of which may be a different data type, but together represent the employee's information as a whole.

Another example is that of a group of items, all of which have the same data type and which need to be kept together for purposes of sorting or searching. A typical data structure provided by programming languages is the **array**, which collects data into a sequence of individually indexed items. Variables can name a data structure in a declaration, as in the C:

```
int a[10];
```

which establishes the variable `a` as the name of an array of ten integer values.

Yet another example is the text file, which is an abstraction that represents a sequence of characters for transfer to and from an external storage device. A text file's structure is independent of the type of storage medium, which can be a magnetic disk, an optical disc (CD or DVD), a solid-state device (flash stick), or even the keyboard and console window.

Like primitive data values, a data structure is an abstraction that hides a group of component parts, allowing the programmer to view them as one thing. Unlike primitive data values, however, data structures provide the programmer with the means of constructing them from their component parts (which can include other data structures as well as primitive values) and also the means of accessing and modifying these components. The different ways of creating and using structured types are examined in Chapter 8.

1.2.3 Data: Unit Abstractions

In a large program, it is useful and even necessary to group related data and operations on these data together, either in separate files or in separate language structures within a file. Typically, such abstractions include access conventions and restrictions that support **information hiding**. These mechanisms

vary widely from language to language, but they allow the programmer to define new data types (data and operations) that hide information in much the same manner as the basic data types of the language. Thus, the unit abstraction is often associated with the concept of an **abstract data type**, broadly defined as a set of data values and the operations on those values. Its main characteristic is the separation of an interface (the set of operations available to the user) from its implementation (the internal representation of data values and operations). Examples of large-scale unit abstractions include the **module** of ML, Haskell, and Python and the **package** of Lisp, Ada, and Java. Another, smaller-scale example of a unit abstraction is the **class** mechanism of object-oriented languages. In this text, we study modules and abstract data types in Chapter 11, whereas classes (and their relation to abstract data types) are studied in Chapter 5.

An additional property of a unit data abstraction that has become increasingly important is its **reusability**—the ability to reuse the data abstraction in different programs, thus saving the cost of writing abstractions from scratch for each program. Typically, such data abstractions represent **components** (operationally complete pieces of a program or user interface) and are entered into a library of available components. As such, unit data abstractions become the basis for language **library** mechanisms (the library mechanism itself, as well as certain standard libraries, may or may not be part of the language itself). The combination of units (their **interoperability**) is enhanced by providing standard conventions for their interfaces. Many interface standards have been developed, either independently of the programming language, or sometimes tied to a specific language. Most of these apply to the class structure of object-oriented languages, since classes have proven to be more flexible for reuse than most other language structures (see the next section and Chapter 5).

When programmers are given a new software resource to use, they typically study its **application programming interface (API)**. An API gives the programmer only enough information about the resource's classes, methods, functions, and performance characteristics to be able to use that resource effectively. An example of an API is Java's Swing Toolkit for graphical user interfaces, as defined in the package `javax.swing`. The set of APIs of a modern programming language, such as Java or Python, is usually organized for easy reference in a set of Web pages called a **doc**. When Java or Python programmers develop a new library or package of code, they create the API for that resource using a software tool specifically designed to generate a doc.

1.2.4 Control: Basic Abstractions

Typical basic control abstractions are those statements in a language that combine a few machine instructions into an abstract statement that is easier to understand than the machine instructions. We have already mentioned the algebraic notation of the arithmetic and assignment expressions, as, for example:

```
SUM = FIRST + SECOND
```

This code fetches the values of the variables `FIRST` and `SECOND`, adds these values, and stores the result in the location named by `SUM`. This type of control is examined in Chapters 7 and 9.

The term **syntactic sugar** is used to refer to any mechanism that allows the programmer to replace a complex notation with a simpler, shorthand notation. For example, the extended assignment operation `x += 10` is shorthand for the equivalent but slightly more complex expression `x = x + 10`, in C, Java, and Python.

1.2.5 Control: Structured Abstractions

Structured control abstractions divide a program into groups of instructions that are nested within tests that govern their execution. They, thus, help the programmer to express the logic of the primary control structures of sequencing, selection, and iteration (loops). At the machine level, the processor executes a sequence of instructions simply by advancing a program counter through the instructions' memory addresses. Selection and iteration are accomplished by the use of **branch instructions** to memory locations other than the next one. To illustrate these ideas, Figure 1.4 shows an LC-3 assembly language code segment that computes the sum of the absolute values of 10 integers in an array named `LIST`. Comments have been added to aid the reader.

```
       LEA  R1, LIST      ; Load the base address of the array (the first cell)
       AND  R2, R2, #0    ; Set the sum to 0
       AND  R3, R3, #0    ; Set the counter to 10 (to count down)
       ADD  R3, R3, #10
WHILE  LDR  R4, R1, #0    ; Top of the loop: load the datum from the current
                          ; array cell
       BRZP INC           ; If it's >= 0, skip next two steps
       NOT  R4, R4        ; It was < 0, so negate it using twos complement
                          ; operations
       ADD  R4, R4, #1
INC    ADD  R2, R2, R4    ; Increment the sum
       ADD  R1, R1, #1    ; Increment the address to move to the next array
                          ; cell
       ADD  R3, R3, #-1   ; Decrement the counter
       BRP  WHILE         ; Goto the top of the loop if the counter > 0
       ST   R2, SUM       ; Store the sum in memory
```

Figure 1.4 An array-based loop in assembly language

If the comments were not included, even a competent LC-3 programmer probably would not be able to tell at a glance what this algorithm does. Compare this assembly language code with the use of the structured `if` and `for` statements in the functionally equivalent C++ or Java code in Figure 1.5.

```
int sum = 0;
for (int i = 0; i < 10; i++){
    int data = list[i];
    if (data < 0)
        data = -data;
    sum += data;
}
```

Figure 1.5 An array-based loop in C++ or Java

Structured selection and loop mechanisms are studied in Chapter 9.

Another structured form of iteration is provided by an **iterator**. Typically found in object-oriented languages, an iterator is an object that is associated with a collection, such as an array, a list, a set, or a tree. The programmer opens an iterator on a collection and then visits all of its elements by running the iterator's methods in the context of a loop. For example, the following Java code segment uses an iterator to print the contents of a list, called `exampleList`, of strings:

```
Iterator<String> iter = exampleList.iterator()
while (iter.hasNext())
    System.out.println(iter.next());
```

The iterator-based traversal of a collection is such a common loop pattern that some languages, such as Java, provide syntactic sugar for it, in the form of an **enhanced `for` loop**:

```
for (String s : exampleList)
    System.out.println(s);
```

We can use this type of loop to further simplify the Java code for computing the sum of either an array or a list of integers, as follows:

```
int sum = 0;
for (int data : list){
    if (data < 0)
        data = -data;
    sum += data;
}
```

Iterators are covered in detail in Chapter 5.

Another powerful mechanism for structuring control is the **procedure**, sometimes also called a **subprogram** or **subroutine**. This allows a programmer to consider a sequence of actions as a single action that can be called or invoked from many other points in a program. Procedural abstraction involves two things. First, a procedure must be defined by giving it a name and associating with it the actions that are to be performed. This is called **procedure declaration**, and it is similar to variable and type declaration, mentioned earlier. Second, the procedure must actually be called at the point where the actions are to be performed. This is sometimes also referred to as procedure **invocation** or procedure **activation**.

As an example, consider the sample code fragment that computes the greatest common divisor of integers `u` and `v`. We can make this into a procedure in Ada with the procedure declaration as given in Figure 1.6.

```
procedure gcd(u, v: in integer; x: out integer) is
    y, t, z: integer;
    begin
        z := u;
        y := v;
        loop
            exit when y = 0;
```

Figure 1.6 An Ada `gcd` procedure *(continues)*

(continued)

```
            t := y;
            y := z mod y;
            z := t;
         end loop;
         x := z;
      end gcd;
```

Figure 1.6 An Ada `gcd` procedure

In this code, we see the procedure header in the first line. Here `u`, `v`, and `x` are **parameters** to the procedure—that is, things that can change from call to call. This procedure can now be **called** by simply naming it and supplying appropriate **actual parameters** or **arguments**, as in:

```
gcd (8, 18, d);
```

which gives `d` the value 2. (The parameter `x` is given the `out` label in line 1 to indicate that its value is computed by the procedure itself and will change the value of the corresponding actual parameter of the caller.)

The system implementation of a procedure call is a more complex mechanism than selection or looping, since it requires the storing of information about the condition of the program at the point of the call and the way the called procedure operates. Such information is stored in a **runtime environment**. Procedure calls, parameters, and runtime environments are all studied in Chapter 10.

An abstraction mechanism closely related to procedures is the **function**, which can be viewed simply as a procedure that returns a value or result to its caller. For example, the Ada code for the `gcd` procedure in Figure 1.6 can more appropriately be written as a function as given in Figure 1.7. Note that the `gcd` function uses a recursive strategy to eliminate the loop that appeared in the earlier version. The use of recursion further exploits the abstraction mechanism of the subroutine to simplify the code.

```
function gcd(u, v: in integer) return integer is
    begin
        if v = 0
            return u;
        else
            return gcd(v, u mod v);
        end if;
    end gcd;
```

Figure 1.7 An Ada `gcd` function

The importance of functions is much greater than the correspondence to procedures implies, since functions can be written in such a way that they correspond more closely to the mathematical abstraction of a function. Thus, unlike procedures, functions can be understood independently of the von Neumann concept of a computer or runtime environment. Moreover, functions can be combined into higher-level abstractions known as **higher-order functions**. Such functions are capable of accepting other functions as arguments and returning functions as values. An example of a higher-order function is a **map**.

This function expects another function and a collection, usually a list, as arguments. The map builds and returns a list of the results of applying the argument function to each element in the argument list. The next example shows how the `map` function is used in Scheme, a dialect of Lisp, to build a list of the absolute values of the numbers in another list. The first argument to `map` is the function `abs`, which returns the absolute value of its argument. The second argument to `map` is a list constructed by the function `list`.

```
(map abs (list 33 -10 66 88 -4))              ; Returns (33 10 66 88 4)
```

Another higher-order function is named `reduce`. Like `map`, `reduce` expects another function and a list as arguments. However, unlike `map`, this function boils the values in the list down to a single value by repeatedly applying its argument function to these values. For example, the following function call uses both `map` and `reduce` to simplify the computation of the sum of the absolute values of a list of numbers:

```
(reduce + (map abs (list 33 -10 66 88 -4))     ; Returns 201
```

In this code, the `list` function first builds a list of numbers. This list is then fed with the `abs` function to the `map` function, which returns a list of absolute values. This list, in turn, is passed with the `+` function (meaning add two numbers) to the `reduce` function. The `reduce` function uses `+` to essentially add up all the list's numbers and return the result.

The extensive use of functions is the basis of the functional programming paradigm and the functional languages mentioned later in this chapter, and is discussed in detail in Chapter 3.

1.2.6 Control: Unit Abstractions

Control can also be abstracted to include a collection of procedures that provide logically related services to other parts of a program and that form a **unit**, or stand-alone, part of the program. For example, a data management program may require the computation of statistical indices for stored data, such as mean, median, and standard deviation. The procedures that provide these operations can be collected into a program unit that can be translated separately and used by other parts of the program through a carefully controlled interface. This allows the program to be understood as a whole without needing to know the details of the services provided by the unit.

Note that what we have just described is essentially the same as a unit-level data abstraction, and is usually implemented using the same kind of module or package language mechanism. The only difference is that here the focus is on the operations rather than the data, but the goals of reusability and library building remain the same.

One kind of control abstraction that is difficult to fit into any one abstraction level is that of parallel programming mechanisms. Many modern computers have several processors or processing elements and are capable of processing different pieces of data simultaneously. A number of programming languages include mechanisms that allow for the parallel execution of parts of programs, as well as providing for synchronization and communication among such program parts. Java has mechanisms for declaring **threads** (separately executed control paths within the Java system) and **processes** (other programs executing outside the Java system). Ada provides the **task** mechanism for parallel execution. Ada's tasks are essentially a unit abstraction, whereas Java's threads and processes are classes and so are structured abstractions, albeit part of the standard `java.lang` package. Other languages provide different levels of parallel abstractions, even down to the statement level. Parallel programming mechanisms are surveyed in Chapter 13.

1.3 Computational Paradigms

Programming languages began by imitating and abstracting the operations of a computer. It is not surprising that the kind of computer for which they were written had a significant effect on their design. In most cases, the computer in question was the von Neumann model mentioned in Section 1.1: a single central processing unit that sequentially executes instructions that operate on values stored in memory. These are typical features of a language based on the von Neumann model: variables represent memory locations, and assignment allows the program to operate on these memory locations.

A programming language that is characterized by these three properties—the sequential execution of instructions, the use of variables representing memory locations, and the use of assignment to change the values of variables—is called an **imperative** language, because its primary feature is a sequence of statements that represent commands, or imperatives.

Most programming languages today are imperative, but, as we mentioned earlier, it is not necessary for a programming language to describe computation in this way. Indeed, the requirement that computation be described as a sequence of instructions, each operating on a single piece of data, is sometimes referred to as the **von Neumann bottleneck**. This bottleneck restricts the ability of a language to provide either parallel computation, that is, computation that can be applied to many different pieces of data simultaneously, or nondeterministic computation, computation that does not depend on order.[1] Thus, it is reasonable to ask if there are ways to describe computation that are less dependent on the von Neumann model of a computer. Indeed there are, and these will be described shortly. Imperative programming languages actually represent only one **paradigm**, or pattern, for programming languages.

Two alternative paradigms for describing computation come from mathematics. The **functional** paradigm is based on the abstract notion of a function as studied in the lambda calculus. The **logic** paradigm is based on symbolic logic. Each of these will be the subject of a subsequent chapter. The importance of these paradigms is their correspondence to mathematical foundations, which allows them to describe program behavior abstractly and precisely. This, in turn, makes it much easier to determine if a program will execute correctly (even without a complete theoretical analysis), and makes it possible to write concise code for highly complex tasks.

A fourth programming paradigm, the **object-oriented** paradigm, has acquired enormous importance over the last 20 years. Object-oriented languages allow programmers to write reusable code that operates in a way that mimics the behavior of objects in the real world; as a result, programmers can use their natural intuition about the world to understand the behavior of a program and construct appropriate code. In a sense, the object-oriented paradigm is an extension of the imperative paradigm, in that it relies primarily on the same sequential execution with a changing set of memory locations, particularly in the implementation of objects. The difference is that the resulting programs consist of a large number of very small pieces whose interactions are carefully controlled and yet easily changed. Moreover, at a higher level of abstraction, the interaction among objects via message passing can map nicely to the collaboration of parallel processors, each with its own area of memory. The object-oriented paradigm has essentially become a new standard, much as the imperative paradigm was in the past, and so will feature prominently throughout this book.

Later in this book, an entire chapter is devoted to each of these paradigms.

[1]Parallel and nondeterministic computations are related concepts; see Chapter 13.

1.4 Language Definition

Documentation for the early programming languages was written in an informal way, in ordinary English. However, as we saw earlier in this chapter, programmers soon became aware of the need for more precise descriptions of a language, to the point of needing formal definitions of the kind found in mathematics. For example, without a clear notion of the meaning of programming language constructs, a programmer has no clear idea of what computation is actually being performed. Moreover, it should be possible to reason mathematically about programs, and to do this requires formal verification or proof of the behavior of a program. Without a formal definition of a language this is impossible.

But there are other compelling reasons for the need for a formal definition. We have already mentioned the need for machine or implementation independence. The best way to achieve this is through standardization, which requires an independent and precise language definition that is universally accepted. Standards organizations such as ANSI (American National Standards Institute) and ISO (International Organization for Standardization) have published definitions for many languages, including C, C++, Ada, Common Lisp, and Prolog.

A further reason for a formal definition is that, inevitably in the programming process, difficult questions arise about program behavior and interaction. Programmers need an adequate way to answer such questions besides the often-used trial-and-error process: it can happen that such questions need to be answered already at the design stage and may result in major design changes.

Finally, the requirements of a formal definition ensure discipline when a language is being designed. Often a language designer will not realize the consequences of design decisions until he or she is required to produce a clear definition.

Language definition can be loosely divided into two parts: **syntax**, or structure, and **semantics**, or meaning. We discuss each of these categories in turn.

1.4.1 Language Syntax

The syntax of a programming language is in many ways like the grammar of a natural language. It is the description of the ways different parts of the language may be combined to form phrases and, ultimately, sentences. As an example, the syntax of the `if` statement in C may be described in words as follows:

> **PROPERTY:** An `if` statement consists of the word "if" followed by an expression inside parentheses, followed by a statement, followed by an optional `else` part consisting of the word "else" and another statement.

The description of language syntax is one of the areas where formal definitions have gained acceptance, and the syntax of all languages is now given using a **grammar**. For example, a grammar rule for the C `if` statement can be written as follows:

```
<if-statement> ::= if (<expression>) <statement>
                   [else <statement>]
```

or (using special characters and formatting):

if-statement → `if` (*expression*) *statement*
[`else` *statement*]

The **lexical structure** of a programming language is the structure of the language's words, which are usually called **tokens**. Thus, lexical structure is similar to spelling in a natural language. In the example of a C `if` statement, the words `if` and `else` are tokens. Other tokens in programming languages include identifiers (or names), symbols for operations, such as `+` and `*` and special punctuation symbols such as the semicolon (`;`) and the period (`.`).

In this book, we shall consider syntax and lexical structure together; a more detailed study can be found in Chapter 6.

1.4.2 Language Semantics

Syntax represents only the surface structure of a language and, thus, is only a small part of a language definition. The semantics, or meaning, of a language is much more complex and difficult to describe precisely. The first difficulty is that "meaning" can be defined in many different ways. Typically, describing the meaning of a piece of code involves describing the effects of executing the code, but there is no standard way to do this. Moreover, the meaning of a particular mechanism may involve interactions with other mechanisms in the language, so that a comprehensive description of its meaning in all contexts may become extremely complex.

To continue with our example of the C `if` statement, its semantics may be described in words as follows (adapted from Kernighan and Richie [1988]):

> An `if` statement is executed by first evaluating its expression, which must have an arithmetic or pointer type, including all side effects, and if it compares unequal to 0, the statement following the expression is executed. If there is an `else` part, and the expression is 0, the statement following the "else" is executed.

This description itself points out some of the difficulty in specifying semantics, even for a simple mechanism such as the `if` statement. The description makes no mention of what happens if the condition evaluates to 0, but there is no `else` part (presumably nothing happens; that is, the program continues at the point after the `if` statement). Another important question is whether the `if` statement is "safe" in the sense that there are no other language mechanisms that may permit the statements inside an `if` statement to be executed without the corresponding evaluation of the `if` expression. If so, then the if-statement provides adequate protection from errors during execution, such as division by zero:

```
if (x != 0) y = 1 / x;
```

Otherwise, additional protection mechanisms may be necessary (or at least the programmer must be aware of the possibility of circumventing the if expression).

The alternative to this informal description of semantics is to use a formal method. However, no generally accepted method, analogous to the use of context-free grammars for syntax, exists here either. Indeed, it is still not customary for a formal definition of the semantics of a programming language to be given at all. Nevertheless, several notational systems for formal definitions have been developed and are increasingly in use. These include **operational semantics**, **denotational semantics**, and **axiomatic semantics**.

Language semantics are implicit in many of the chapters of this book, but semantic issues are more specifically addressed in Chapters 7 and 11. Chapter 12 discusses formal methods of semantic definition, including operational, denotational, and axiomatic semantics.

1.5 Language Translation

For a programming language to be useful, it must have a **translator**—that is, a program that accepts other programs written in the language in question and that either executes them directly or transforms them into a form suitable for execution. A translator that executes a program directly is called an **interpreter**, while a translator that produces an equivalent program in a form suitable for execution is called a **compiler**.

As shown in Figure 1-8, interpretation is a one-step process, in which both the program and the input are provided to the interpreter, and the output is obtained.

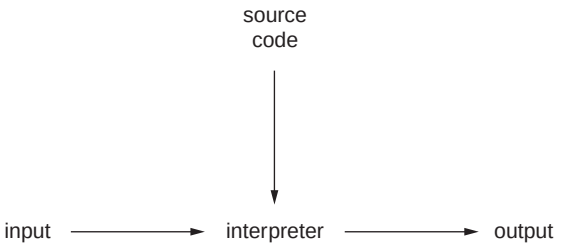

Figure 1.8 The interpretation process

An interpreter can be viewed as a simulator for a machine whose "machine language" is the language being translated.

Compilation, on the other hand, is at least a two-step process: the original program (or **source program**) is input to the compiler, and a new program (or **target program**) is output from the compiler. This target program may then be executed, if it is in a form suitable for direct execution (i.e., in machine language). More commonly, the target language is assembly language, and the target program must be translated by an **assembler** into an object program, and then **linked** with other object programs, and **loaded** into appropriate memory locations before it can be executed. Sometimes the target language is even another programming language, in which case a compiler for that language must be used to obtain an executable object program.

Alternatively, the target language is a form of low-level code known as **byte code**. After a compiler translates a program's source code to byte code, the byte code version of the program is executed by an interpreter. This interpreter, called a **virtual machine**, is written differently for different hardware architectures, whereas the byte code, like the source language, is machine-independent. Languages such as Java and Python compile to byte code and execute on virtual machines, whereas languages such as C and C++ compile to native machine code and execute directly on hardware.

The compilation process can be visualized as shown in Figure 1.9.

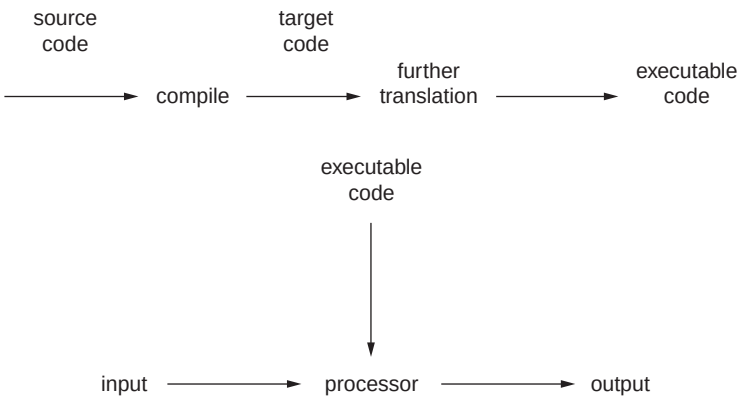

Figure 1.9 The compilation process

It is important to keep in mind that a language and the translator for that language are two different things. It *is* possible for a language to be defined by the behavior of a particular interpreter or compiler (a so-called **definitional** translator), but this is not common (and may even be problematic, in view of the need for a formal definition, as discussed in the last section). More often, a language definition exists independently, and a translator may or may not adhere closely to the language definition (one hopes the former). When writing programs one must always be aware of those features and properties that depend on a specific translator and are not part of the language definition. There are significant advantages to be gained from avoiding nonstandard features as much as possible.

A complete discussion of language translation can be found in compiler texts, but we will examine the basic front end of this process in Chapters 6–10.

1.6 The Future of Programming Languages

In the 1960s, some computer scientists dreamed of a single universal programming language that would meet the needs of all computer users. Attempts to design and implement such a language, however, resulted in frustration and failure. In the late 1970s and early 1980s, a different dream emerged—a dream that programming languages themselves would become obsolete, that new **specification languages** would be developed that would allow computer users to just say what they wanted to a system that would then find out how to implement the requirements. A succinct exposition of this view is contained in Winograd [1979]:

> Just as high-level languages enabled the programmer to escape from the intricacies of a machine's order code, higher level programming systems can provide help in understanding and manipulating complex systems and components. We need to shift our attention away from the detailed specification of algorithms, towards the description of the properties of the packages and objects with which we build. A new generation of programming tools will be based on the attitude that what we say in a programming system should be primarily declarative, not imperative: the fundamental use of a programming system is not in creating sequences of instructions for accomplishing tasks (or carrying out algorithms), but in expressing and manipulating descriptions of computational processes and the objects on which they are carried out. (Ibid., p. 393)

In a sense, Winograd is just describing what logic programming languages attempt to do. As you will see in Chapter 4, however, even though these languages can be used for quick prototyping, programmers still need to specify algorithms step by step when efficiency is needed. Little progress has been made in designing systems that can on their own construct algorithms to accomplish a set of given requirements.

Programming has, thus, not become obsolete. In a sense it has become even more important, since it now can occur at so many different levels, from assembly language to specification language. And with the development of faster, cheaper, and easier-to-use computers, there is a tremendous demand for more and better programs to solve a variety of problems.

What's the future of programming language design? Predicting the future is notoriously difficult, but it is still possible to extrapolate from recent trends. Two of the most interesting perspectives on the evolution of programming languages in the last 20 years come from a pair of second-generation Lisp programmers, Richard Gabriel and Paul Graham.

In his essay "The End of History and the Last Programming Language" [Gabriel 1996], Gabriel is puzzled by the fact that very high-level, mathematically elegant languages such as Lisp have not caught on in industry, whereas less elegant and even semantically unsafe languages such as C and C++ have become the standard. His explanation is that the popularity of a programming language is much more a function of the context of its use than of any of its intrinsic properties. To illustrate this point, he likens the spread of C in the programming community to that of a virus. The simple footprint of the C compiler and runtime environment and its connection to the UNIX operating system has allowed it to spread rapidly to many hardware platforms. Its conventional syntax and lack of mathematical elegance have appealed to a very wide range of programmers, many of whom may not necessarily have much mathematical sophistication. For these reasons, Gabriel concludes that C will be the ultimate survivor among programming languages well into the future.

Graham, writing a decade later in his book *Hacker and Painters* [Graham 2004] sees a different trend developing. He believes that major recent languages, such as Java, Python, and Ruby, have added features that move them further away from C and closer to Lisp. However, like C in an earlier era, each of these languages has quickly migrated into new technology areas, such as Web-based client/server applications and mobile devices. What then of Lisp itself? Like most writers on programming languages, Graham classifies them on a continuum, from fairly low level (C) to fairly high level (Java, Python, Ruby). But he then asks two interesting questions: If there is a range of language levels, which languages are at the highest level? And if there is a language at the highest level, and it still exists, why wouldn't people prefer to write their programs in it? Not surprisingly, Graham claims that Lisp, after 50 years, always has been and still is the highest-level language. He then argues, in a similar manner to Gabriel, that Lisp's virtues have been recognized only by the best programmers and by the designers of the aforementioned recent languages. However, Graham believes that the future of Lisp may lie in the rapid development of server-side applications.

Figure 1.10 shows some statistics on the relative popularity of programming languages since 2000. The statistics, which include the number of posts on these languages on comp.lang newsgroups for the years 2009, 2003, and 2000, lend some support to Graham's and Gabriel's analyses. (Comp newsgroups, originally formed on Usenet, provide a forum for discussing issues in technology, computing, and programming.)

```
    Mar 2009 (100d)       Feb 2003 (133 d)     Jan 2000 (365d)
    news.tuwien.ac.at     news.individual.net  tele.dk
    posts language        posts language       posts language
 1 14110 python           59814 java           229034 java
 2 13268 c                44242 c++            114769 basic
 3  9554 c++              27054 c              113001 perl
 4  9057 ruby             24438 python         102261 c++
 5  9054 java             23590 perl           79139 javascript
 6  5981 lisp             18993 javascript     70135 c
```

Figure 1.10 Popularity of programming languages (source: *www.complang.tuwien.ac.at/anton/comp.lang-statistics/*)

One thing is clear. As long as new computer technologies arise, there will be room for new languages and new ideas, and the study of programming languages will remain as fascinating and exciting as it is today.

Exercises

1.1 Explain why von Neumann's idea of storing a program in computer memory represented an advance for the operators of computers.

1.2 State a difficulty that machine language programmers faced when **(a)** translating their ideas into machine code, and **(b)** loading their code by hand into computer memory.

1.3 List at least three ways in which the use of assembly language represented an improvement for programmers over machine language.

1.4 An abstraction allows programmers to say more with less in their code. Justify this statement with two examples.

1.5 ALGOL was one of the first programming languages to achieve machine independence, but not independence from the von Neumann model of computation. Explain how this is so.

1.6 The languages Scheme, C++, Java, and Python have an integer data type and a string data type. Explain how values of these types are abstractions of more complex data elements, using at least one of these languages as an example.

1.7 Explain the difference between a data structure and an abstract data type (ADT), using at least two examples.

1.8 Define a recursive factorial function in any of the following languages (or in any language for which you have a translator): **(a)** Scheme, **(b)** C++, **(c)** Java, **(d)** Ada, or **(e)** Python.

1.9 Assembly language uses branch instructions to implement loops and selection statements. Explain why a `for` loop and an `if` statement in high-level languages represent an improvement on this assembly language technique.

1.10 What is the difference between the use of an index-based loop and the use of an iterator with an array? Give an example to support your answer.

1.11 List three reasons one would package code in a procedure or function to solve a problem.

1.12 What role do parameters play in the definition and use of procedures and functions?

1.13 In what sense does recursion provide additional abstraction capability to function definitions? Give an example to support your answer.
1.14 Explain what the `map` function does in a functional language. How does it provide additional abstraction capability in a programming language?
1.15 Which three properties characterize imperative programming languages?
1.16 How do the three properties in your answer to question 1.15 reflect the von Neumann model of computing?
1.17 Give two examples of lexical errors in a program, using the language of your choice.
1.18 Give two examples of syntax errors in a program, using the language of your choice.
1.19 Give two examples of semantic errors in a program, using the language of your choice.
1.20 Give one example of a logic error in a program, using the language of your choice.
1.21 Java and Python programs are translated to byte code that runs on a virtual machine. Discuss the advantages and disadvantages of this implementation strategy, as opposed to that of C++, whose programs translate to machine code.

Notes and References

The quote from A. N. Whitehead in Section 1.1 is in Whitehead [1911]. An early description of the von Neumann architecture and the use of a program stored as data to control the execution of a computer is in Burks, Goldstine, and von Neumann [1947]. A gentle introduction to the von Neumann architecture, and the evolution of computer hardware and programming languages is in Schneider and Gersting [2010].

References for the major programming languages used or mentioned in this text are as follows. The LC-3 machine architecture, instruction set, and assembly language are discussed in Patt and Patel [2003]. The history of FORTRAN is given in Backus [1981]; of Algol60 in Naur [1981] and Perlis [1981]; of Lisp in McCarthy [1981], Steele and Gabriel [1996], and Graham [2002]; of COBOL in Sammet [1981]; of Simula67 in Nygaard and Dahl [1981]; of BASIC in Kurtz [1981]; of PL/I in Radin [1981]; of SNOBOL in Griswold [1981]; of APL in Falkoff and Iverson [1981]; of Pascal in Wirth [1996]; of C in Ritchie [1996]; of C++ in Stroustrup [1994] [1996]; of Smalltalk in Kay [1996]; of Ada in Whitaker [1996]; of Prolog in Colmerauer and Roussel [1996]; of Algol68 in Lindsey [1996]; and of CLU in Liskov [1996]. A reference for the C programming language is Kernighan and Ritchie [1988]. The latest C standard is ISO 9899 [1999]. C++ is described in Stroustrup [1994] [1997], and Ellis and Stroustrup [1990]; an introductory text is Lambert and Nance [2001]; the international standard for C++ is ISO 14882-1 [1998]. Java is described in many books, including Horstmann [2006] and Lambert and Osborne [2010]; the Java language specification is given in Gosling, Joy, Steele, and Bracha [2005]. Ada exists in three versions: The original is sometimes called Ada83, and is described by its reference manual (ANSI-1815A [1983]); newer versions are Ada95 and Ada2005[2], and are described by their international standard (ISO 8652 [1995, 2007]). A standard text for Ada is Barnes [2006]. An introductory text on Python is Lambert [2010]. Common Lisp is presented in Graham [1996] and Seibel [2005]. Scheme is described in Dybvig [1996] and Abelson and Sussman [1996]; a language definition can be found in

[2]Since Ada95/2005 is an extension of Ada83, we will indicate only those features that are specifically Ada95/2005 when they are not part of Ada83.

Abelson et al. [1998]. Haskell is covered in Hudak [2000] and Thompson [1999]. The ML functional language (related to Haskell) is covered in Paulson [1996] and Ullman [1997]. The standard reference for Prolog is Clocksin and Mellish [1994]. The logic paradigm is discussed in Kowalski [1979], and the functional paradigm in Backus [1978] and Hudak [1989]. Smalltalk is presented in Lambert and Osborne [1997]. Ruby is described in Flanagan and Matsumoto [2008] and in Black [2009]. Erlang is discussed in Armstrong [2007].

Language translation techniques are described in Aho, Lam, Sethi, and Ullman [2006] and Louden [1997].

Richard Gabriel's essay on the last programming language appears in Gabriel [1996], which also includes a number of interesting essays on design patterns. Paul Graham's essay on high-level languages appears in Graham [2004], where he also discusses the similarities between the best programmers and great artists.